

Neural networks.

Built from first principles.

From one weighted sum to a network that learns.
Understand each prediction, gradient, and update.

🕒 18-22 min core reading · 25-35 min practice · 12 short sections

A neural network is a function built from small, connected calculations. Training adjusts its numbers so its outputs better match examples. We will open that function, follow one movie through it, and train a network small enough to inspect completely.

WHAT YOU WILL UNDERSTAND

The main question: how do weights, biases, layers, and activations turn movie features into a prediction, and how does learning change that prediction?

The practical path: calculate a neuron, follow a forward pass, trace the loss backward, and compare a linear classifier with a small neural network.

Reading path: twelve short sections, three diagrams, real browser training, and a Python workbook. Chapters 6, 8, and 13 provide optional refreshers on matrices, derivatives, and classification.

1. Start with a pattern that needs more than a line

Imagine a viewer who likes an **adventure or a romance, but not a mixture of both**. Movies with neither also receive a dislike. This deliberately simplified preference is called **exclusive OR (XOR)**: the label is 1 when exactly one input is 1.

Fictional movie	Adventure x_1	Romance x_2	Like y
Moonlight Trail	1	0	1
Letters at Dawn	0	1	1
Hearts on the Run	1	1	0
Quiet Notebook	0	0	0

These four movies form a new teaching dataset; they do not describe real users or change the earlier catalog. A feature value of 0 means the genre is absent here, not missing data. All four labels are visible and used for training.

Plot the inputs at the corners of a square: the liked movies occupy opposite corners. No single straight line can separate both liked corners from both disliked corners. Logistic regression on these two raw features has a straight decision boundary. A hidden layer with a nonlinear activation lets us build a more flexible score.

▼ Go deeper: prove the limitation, and see another way to solve it

For a linear score $z = w_1x_1 + w_2x_2 + b$ with “like” when $z \geq 0$, the two dislikes require $b < 0$ and $w_1 + w_2 + b < 0$. The two likes require $w_1 + b \geq 0$ and $w_2 + b \geq 0$. Adding the latter two gives $w_1 + w_2 + 2b \geq 0$; adding the former two gives the same expression strictly below zero. They cannot all hold.

A neural network is not the only solution. Hand-engineering the interaction x_1x_2 makes the score $x_1 + x_2 - 2x_1x_2 - 0.5$ separate the four rows. A decision tree could also express the rule. The useful neural-network idea is that intermediate features can be learned jointly with the final predictor. This example does not establish that a neural network is the best recommender.

2. Build one neuron from weights and a bias

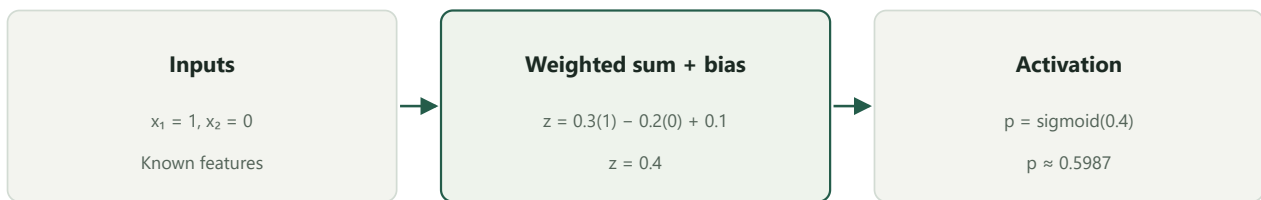
A **neuron**, or unit, first computes a weighted sum plus a bias, then applies an activation function:

$$z = w_1x_1 + w_2x_2 + b, \quad h = \phi(z).$$

The **weights** determine how much each input contributes. The **bias** shifts the sum even when the inputs are zero. Together they are learned **parameters**. The activation ϕ is a chosen function; h is its output. A weighted sum plus a bias is technically **affine**, although “linear layer” is common shorthand.

With $x = (1, 0)$, weights $(0.3, -0.2)$, and bias 0.1, the sum is 0.4. Applying the sigmoid gives approximately 0.5987. A negative romance weight lowers this unit’s score when that input changes from 0 to 1, holding everything else fixed. In a larger network, one weight alone does not explain the whole decision.

➤ A neuron is a small calculation



Weights control contributions; the bias shifts the starting point.

Probability is a model estimate. The 0.5 decision threshold is a separate choice.

Figure 1. Weight each input, add a bias, then apply an activation. With $x = (1, 0)$, weights $(0.3, -0.2)$, and bias 0.1 , the sigmoid output is about 0.5987 .

The name “neuron” is loosely inspired by biology. Our unit is an arithmetic operation; it does not possess a movie concept, a memory of a scene, or human understanding.

3. Give the network a nonlinear activation

An activation transforms the sum. **Nonlinear** means the transformation is not simply one straight-line rule across all inputs.

Activation	Rule and range	What to remember
Identity	$\phi(z) = z$; any real value	Often the final output for regression
Sigmoid	$\sigma(z) = 1/(1 + e^{-z})$; between 0 and 1	Useful for a binary probability output
Tanh	$\tanh(z)$; between -1 and 1	Our hidden activation; zero-centered and smooth
ReLU	$\max(0, z)$; zero or positive	Keeps positive values and sets negative ones to zero

Without nonlinear hidden activations, stacking affine layers gives another affine transformation. For example, $3(2x + 1) + 4 = 6x + 7$: extra steps, same kind of function. Tanh or ReLU changes this limitation. The [Dive into Deep Learning explanation of hidden layers](#) develops this distinction.

Sigmoid is nonlinear too, but applying it **only to a linear output score** still leaves a straight boundary at $p = 0.5$, because $\sigma(z) \geq 0.5$ exactly when $z \geq 0$. The location of the nonlinearity matters.

Sigmoid and tanh (hyperbolic tangent) flatten for large-magnitude inputs; their derivatives become small there. ReLU has slope 0 for negative inputs and 1 for positive inputs, with a convention needed at exactly zero. These gradient properties affect training. Tanh is chosen here for a compact, smooth demonstration, not as a universal best activation.

4. Connect units into layers

A **dense**, or fully connected, layer gives every output unit a connection from every input to that layer. Our **multilayer perceptron (MLP)** has two input features, three tanh hidden units, and one sigmoid output. Information moves forward without a recurrent loop, so this is a **feedforward** network.

The **hidden layer** produces intermediate features. “Hidden” means we do not supply target labels for those intermediate values; we can still inspect them. **Width** is the number of units in a layer. **Depth** counts successive learned transformations. Here there are two parameterized layers: hidden and output. The input features are often drawn as an input layer but perform no learned calculation themselves.

➤ **Two inputs. Three hidden units. One output.**

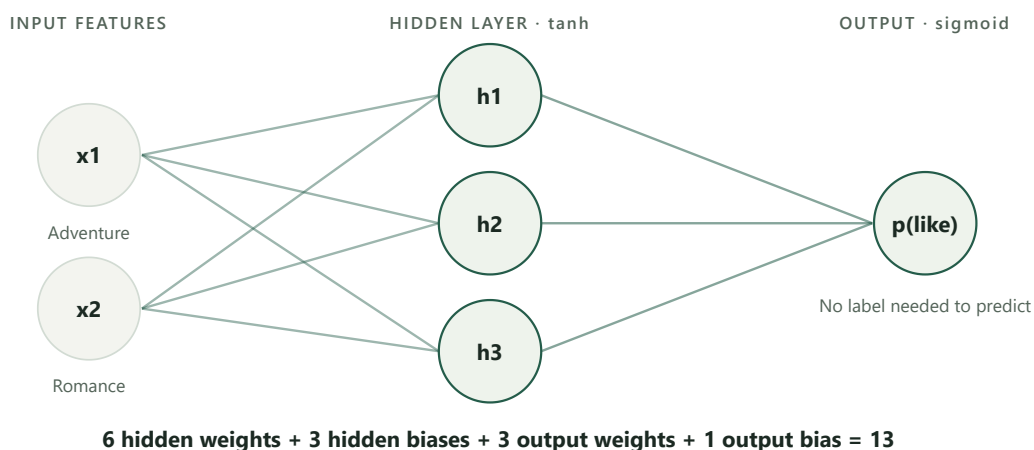


Figure 2. Every input connects to every hidden unit, and every hidden unit connects to the output. Each of the four computed units has a bias. Hidden activations become the output layer’s inputs.

Count parameters, not just circles: the hidden layer has $2 \times 3 = 6$ weights and 3 biases. The output has 3 weights and 1 bias. The total is **13 parameters**. More parameters can increase capacity, but they also require suitable data and training; they do not guarantee a better model.

▼ Go deeper: matrix shapes and batch computation

Using column vectors for one movie:

$$\mathbf{h} = \tanh(W\mathbf{x} + \mathbf{b}), \quad z = \mathbf{v}^\top \mathbf{h} + c, \quad p = \sigma(z).$$

Here $\mathbf{x}$ has shape 2×1 , W is 3×2 , $\mathbf{b}$ and $\mathbf{h}$ are 3×1 , $\mathbf{v}$ is 3×1 , and z, c, p are scalars. Tanh acts separately on each coordinate.

For a batch with movies as rows, X has shape $B \times 2$ and $H = \tanh(XW^\top + \mathbf{b}^\top)$ has shape $B \times 3$.

Broadcasting repeats the bias across rows; it does not create a different learned bias per movie. The output has shape $B \times 1$. A tensor is a numerical array with one or more axes; scalars are order-zero tensors. Matrix notation lets hardware process many examples together, while the workbook uses loops so each operation stays visible.

5. Follow a complete forward pass

Forward propagation evaluates the network using its current parameters. For Moonlight Trail, $x = (1, 0)$. Our fixed starting parameters give:

Hidden unit	(w_1, w_2, b)	Sum a	Activation $h = \tanh(a)$
h_1	$(0.6, -0.4, 0.1)$	0.7000	0.6044
h_2	$(-0.3, 0.8, -0.2)$	-0.5000	-0.4621
h_3	$(0.7, 0.5, 0.05)$	0.7500	0.6351

The output weights are $(0.4, -0.5, 0.3)$ and its bias is -0.1 . Combining the hidden outputs gives:

$$z = 0.4h_1 - 0.5h_2 + 0.3h_3 - 0.1 \approx 0.5634, \quad p = \sigma(z) \approx 0.6372.$$

This final pre-sigmoid score z is the **logit**. A threshold of 0.5 turns the probability into “like.” The known label $y = 1$ is not needed for this calculation; it is needed afterward to measure the error. These starting weights are specified for reproducibility, not learned movie knowledge.

Do not confuse an activation with a parameter. h_1 changes when a different movie passes through the same weights. A weight changes only when we update the model. Values above are rounded for reading; training uses full precision.

6. Turn prediction error into a loss

A **loss function** assigns a numerical penalty to a prediction and its known target. For binary labels we use binary cross-entropy, also called log loss, with natural logarithms:

$$\ell(p, y) = -[y \ln p + (1 - y) \ln(1 - p)].$$

For Moonlight Trail, $y = 1$ and $p \approx 0.6372$, so the loss is $-\ln p \approx 0.4506$. A confident wrong answer receives a larger penalty than an uncertain one. We train on the **mean** loss across all four movies. At initialization that mean is about **0.7008**, not the single movie’s 0.4506.

The output and loss must match the task. A rating regressor can use an unrestricted scalar output with squared error. A binary like/dislike classifier uses one sigmoid with binary cross-entropy. Predicting one of several mutually exclusive categories often uses a **softmax**, which normalizes several logits into probabilities summing to 1, with categorical cross-entropy.

Accuracy counts thresholded decisions. Loss also responds when an already correct answer becomes more confident. That is why a network can remain at 100% training accuracy while its training loss continues to decrease. Neither quantity establishes calibrated probabilities or accuracy on unseen viewers.

▼ Go deeper: softmax, multilabel outputs, and stable loss arithmetic

For K exclusive classes, softmax gives $p_k = e^{z_k} / \sum_j e^{z_j}$. With logits $(2, 1, 0)$, its probabilities are approximately $(0.6652, 0.2447, 0.0900)$. The loss for the correct class t is $-\ln p_t$. Compute softmax after subtracting the maximum logit to avoid overflowing exponentials; this does not change the probabilities.

A movie can have multiple genres at once. Predicting those genre tags is **multilabel** classification, typically using a separate sigmoid per tag rather than forcing all tag probabilities to sum to 1. Our input genre flags are known; the single output predicts a different property, the viewer's like label.

The workbook computes binary loss directly from its logit: $\max(z, 0) - yz + \ln(1 + e^{-|z|})$. This is equivalent to the probability formula but avoids taking the logarithm of a sigmoid rounded to zero or one. A confidently wrong $z = 1000$ with $y = 0$ therefore yields a finite loss near 1000. Stable arithmetic changes the implementation, not the training objective.

7. Trace the error backward with the chain rule

Backpropagation calculates how the loss changes with each parameter. It applies the chain rule from Chapter 8, reusing intermediate values from the forward pass. It computes gradients; an optimizer later uses them to update parameters. The D2L treatment of forward and backward propagation explains this computational-graph view.

For our sigmoid output with binary cross-entropy, the derivative with respect to the output logit simplifies to $\delta = p - y$. For Moonlight Trail, $\delta \approx -0.3628$. With $z = \sum_j v_j h_j + c$, this gives:

$$\frac{\partial \ell}{\partial v_j} = \delta h_j, \quad \frac{\partial \ell}{\partial c} = \delta.$$

To reach a hidden weight, continue through $h_j = \tanh(a_j)$ and $a_j = \sum_i W_{ji} x_i + b_j$:

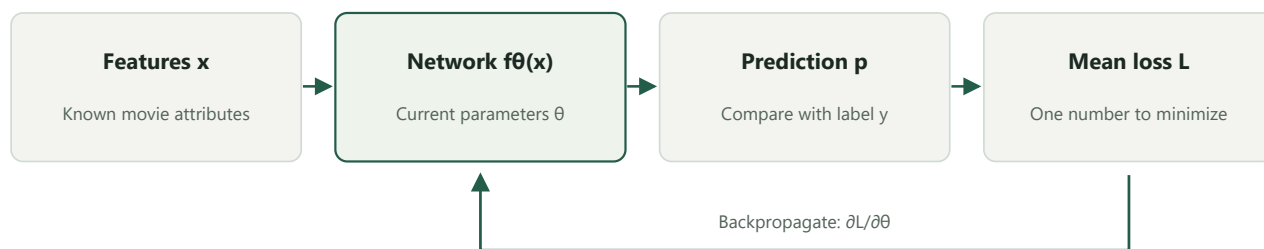
$$\frac{\partial \ell}{\partial W_{ji}} = \delta v_j (1 - h_j^2) x_i.$$

Read it as a path: **output error** × **output connection** × **activation slope** × **input**. For the first hidden unit's adventure weight, the single-movie gradient is approximately

$$(-0.3628)(0.4)(1 - 0.6044^2)(1) \approx -0.0921.$$

Its negative sign says a small increase in that weight would locally lower **this movie's** loss. The romance input is zero, so this movie contributes zero gradient to the same unit's romance weight. Other movies can still update that weight.

➤ Prediction goes forward; gradients flow backward



Optimizer: $\theta_{\text{new}} = \theta_{\text{old}} - \text{learning rate} \times \text{gradient}$

Compute all gradients at the old parameters before updating them together.

Figure 3. Forward propagation computes a prediction and loss. Backpropagation applies the chain rule to calculate gradients. The optimizer updates parameters for the next pass. Inference uses only the forward path.

▼ Go deeper: batch gradients, a numerical check, and automatic differentiation

For the mean batch loss $L = (1/B) \sum_r \ell_r$, average each parameter's per-row derivatives. At our initial parameters, the first hidden adventure weight has a **mean gradient of +0.025594** across all four movies, even though Moonlight Trail alone contributes -0.092106 . A learning rate of 0.5 updates it from 0.6 to approximately 0.6 — $0.5(0.025594) = 0.587203$. Improving the batch objective need not improve every individual row at every step.

Compute all gradients from the same old parameters before changing any weight. Using an already-updated output weight while calculating hidden derivatives would mix two different forward passes.

The workbook checks each analytic derivative against a central finite difference: $(L(\theta + \epsilon e_i) - L(\theta - \epsilon e_i)) / (2\epsilon)$, with $\epsilon = 10^{-5}$. This is a diagnostic approximation, not its training algorithm. Frameworks usually use **automatic differentiation**: derivative rules applied to recorded operations. Backpropagation is the reverse-mode procedure used to efficiently obtain gradients of a scalar loss with respect to many parameters. It is not repeated trial-and-error perturbation of every weight.

8. Update parameters, then predict again

Gradient descent takes a step opposite the gradient:

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla_{\theta} L.$$

Here θ stands for all weights and biases, and η is the **learning rate**. We choose the architecture, activation, learning rate, and training duration; these are **hyperparameters**. Training learns the weights and biases. A gradient describes a local slope, so a larger step does not automatically produce more improvement.

A **batch** is the examples used in one update. An **epoch** is one pass through the training dataset. Our batch contains all four movies, so one optimizer step is one epoch. With 1,000 rows and batches of 100, one epoch normally involves ten updates. A **mini-batch** uses a subset of the training data at each step.

During **inference**, keep the learned parameters fixed and perform the forward calculation. No known like label, loss, or gradient update is required. Reading a new movie does not automatically train the network.

We start hidden units with different weights to avoid making them identical copies that receive identical learning signals. Large networks commonly use random, scale-aware initialization; our unequal starting values are fixed so the example is repeatable. Starting biases at zero is often fine and is different from making all hidden weights identical. Training practice, initialization, regularization, and debugging are developed in Chapter 21.

9. Train and inspect the tiny network

First inspect the untrained network and open Moonlight Trail's trace. Train one step and look at the parameter changes. Then train to 2,000 steps. Reset or switch to the linear model and give it the same step budget. Compare the loss and the four probabilities, not just the rounded decisions.

FOLLOW THE NUMBERS.

Real local training · 4 fictional movies

Same preferences. Different capacity.

A viewer likes exactly one of adventure and romance. Compare a sigmoid of a linear score with a $2 \rightarrow 3 \rightarrow 1$ tanh network. Every row below is training data; there is no test score.

Fixed initialization, learning rate 0.5, all four rows in each update:

Model / steps	Mean training loss	Training accuracy
Linear score / 0	0.7000	25%
Linear score / 2000	0.6931	50%
Tanh network / 0	0.7008	25%
Tanh network / 2000	0.0027	100%

After 2,000 steps, the tanh network's probabilities for N1-N4 are approximately 0.9968, 0.9968, 0.0043, and 0.0003. The linear model approaches 0.5 for all four. These are fitted training predictions, not calibrated probabilities for real viewers.

Initial N1 trace: hidden sums (0.7, -0.5, 0.75), hidden activations (0.6044, -0.4621, 0.6351), logit 0.5634, probability 0.6372, loss 0.4506. The single-row first-weight gradient is -0.0921; the full-batch mean gradient is +0.0256.

At the default learning rate of 0.5, the tanh network reaches a mean training loss of approximately **0.0027** after 2,000 steps and classifies all four training rows correctly. The linear model approaches a loss of **0.6931**, corresponding to probability 0.5 everywhere. This is a representation lesson on a tiny constructed pattern, not a production comparison.

Try another learning rate. The number of updates is only one part of the training setup. We use no regularization, early stopping, validation-based selection, or test set here. All training runs execute locally; no model service or API is involved.

10. Rebuild the calculations in Python

The three-file workbook contains `network.py`, `test_network.py`, and `README.md`. It implements both models, stable loss, manual backpropagation, full-batch updates, and a recorded loss curve using only the Python 3.10+ standard library.

```
py network.py --architecture hidden --steps 2000 --rate 0.5
py network.py --architecture linear --steps 2000 --rate 0.5
py network.py --architecture hidden --steps 1
py -m unittest -v test_network.py
```

Use `python` or `python3` if needed. The commands print JSON without writing files or making network calls. The report includes parameters, per-movie forward values, mean gradients, and a loss history sampled every 20 steps plus the final step. Invalid arguments return exit code 2. Read `forward`, `sample_gradient`, `batch`, and `train` in that order.

Exercise 1: calculate one sigmoid unit

For Letters at Dawn, $x = (0, 1)$, use weights $(0.3, -0.2)$ and bias 0.1. Calculate the logit, approximate probability, and decision at threshold 0.5. Is it correct for $y = 1$?

▼ Reveal the reasoning

The logit is $0.3(0) - 0.2(1) + 0.1 = -0.1$. $\text{Sigmoid}(-0.1) \approx 0.4750$, so the decision is dislike. The label is like, so this prediction is incorrect. Its binary cross-entropy is $-\ln(0.4750) \approx 0.7444$.

Exercise 2: count a wider network

Replace the three hidden units with five, keeping two inputs and one output. Count all trainable weights and biases. Does increasing the count prove that unseen recommendations improve?

▼ Reveal the reasoning

The hidden layer has $2 \times 5 + 5 = 15$ parameters. The output has $5 + 1 = 6$. Total: 21. More capacity may help represent a pattern, but performance on unseen movies needs suitable held-out evaluation; capacity alone proves no improvement.

Exercise 3: distinguish a row gradient from the batch gradient

The first hidden adventure weight is 0.6. Moonlight Trail's gradient is -0.092106 , but the mean gradient across the training batch is $+0.025594$. With learning rate 0.5 and full-batch training, which way does the weight move, and what is its new value?

▼ Reveal the reasoning

Use the mean gradient: $0.6 - 0.5 \times 0.025594 = 0.587203$, so the weight decreases. Using only the negative single-row gradient would instead increase it to about 0.646053. Those are different training updates, not contradictory derivatives.

11. Check your understanding

01 What does a neuron compute in this chapter?

- A. A weighted sum plus a bias, followed by an activation.
- B. A stored sentence about the movie.
- C. An average of every label in the database.

Answer: A. Inputs are multiplied by learned weights, a learned bias shifts the sum, and an activation transforms it.

02 Why do hidden layers need nonlinear activations?

- A. To make every probability exactly 0 or 1.
- B. Without them, stacked affine layers collapse into one affine transformation.
- C. To avoid having any parameters.

Answer: B. Extra affine layers alone do not create a nonlinear decision boundary on the original features. The hidden nonlinearity changes what the network can represent.

03 How many trainable parameters are in our dense $2 \rightarrow 3 \rightarrow 1$ network, including biases?

- A. 6.
- B. 9.
- C. 13.

Answer: C. The hidden layer has 2×3 weights and 3 biases. The output has 3 weights and 1 bias: $6 + 3 + 3 + 1 = 13$.

04 What is the relationship between backpropagation and gradient descent?

- A. Both are names for storing predictions.
- B. Backpropagation computes gradients; gradient descent uses them to update parameters.
- C. Backpropagation changes the training labels.

Answer: B. The chain rule gives derivatives of loss with respect to each parameter. The optimizer then takes a step, such as $\theta - \text{learning rate} \times \text{gradient}$.

05 The network fits all four teaching movies. What have we established?

- A. It will recommend unseen movies perfectly.
- B. Its probabilities are calibrated for real viewers.
- C. It can fit this small training pattern; generalization needs separate evidence.

Answer: C. All four rows were used to train. There are no held-out users or movies here, and binary genre flags leave out most information needed for useful recommendations.

12. Connect learned representations to larger models

The vector of hidden activations is a **representation** of the input. The output layer uses that representation to solve the training task. Its coordinates need not have neat human meanings such as “adventure detector”: multiple units and connections can jointly encode a useful pattern.

More layers compose more transformations. Neural networks underlie image encoders, text embeddings, and LLMs, but those systems add architectures, objectives, and training procedures far beyond this tiny MLP. A trained predictor also does not replace catalog filters, retrieval, or application rules; those components serve different purposes in the movie assistant.

▼ Go deeper: expressive capacity is not a promise of learning

Approximation results describe what sufficiently large networks with suitable nonlinear activations can represent under stated conditions. They do not guarantee that a chosen optimizer finds those parameters, that the network needs little data, or that it generalizes beyond the training distribution. The Deep Learning chapter on feedforward networks distinguishes representation, optimization, and generalization.

Our four rows exhaust the binary combinations, but they are not independent observations of real viewer behavior. There are no unseen viewers, noisy preferences, held-out movies, or additional features. Perfect fit here demonstrates capacity and working optimization on this example. A real recommendation system needs a separate data and evaluation design, as Chapters 12-15 explain.

You should now be able to explain one prediction and one parameter update without treating the model as a black box. **Chapter 21: Training and debugging neural networks** will turn that arithmetic into a practical training workflow: initialization, optimization, generalization, and diagnosing failed learning.

▼ Quick-reference glossary

Term	Meaning in this chapter
Weight / bias	Learned input multiplier / learned additive offset
Activation	Function applied to a unit's pre-activation sum; also its resulting value
Hidden layer / representation	Intermediate computed units / their feature values
MLP / dense layer	Feedforward network of connected units / all inputs connect to every output unit
Logit / sigmoid	Binary output score before normalization / mapping from score to probability
Forward pass / inference	Evaluate current parameters / use the fixed model to predict
Loss / gradient	Error objective / its derivatives with respect to parameters
Backpropagation / optimizer	Compute gradients through the network / use them to update parameters
Batch / epoch	Examples in an update / one pass through the training data
Parameter / hyperparameter	Number learned during training / setting selected for the training process

Sources and further reading

Dive into Deep Learning's hidden-layer and backpropagation chapters, and Deep Learning's feedforward-network chapter, are linked beside the related concepts. The movie scenario, fixed parameters, workbook, diagrams, and numerical experiments are original teaching material.